Darth Invaders

Proposed Changes - Documentation Only - 17 August 2026

> **Scope:** This is an implementation proposal based on a focused inspection of the startup path, game loop, input handling, score persistence, and project documentation. It does not modify the application.

Executive Summary

The game has a clear split between Swing rendering/input and a dedicated game-calculation thread. The first changes should make that split safe and predictable across restart, then make persistence and onboarding reliable. Feature work should follow only after those foundations are covered by automated checks.

Priority	Change	Reason	Acceptance Check
P0	Manage calculator-thread shutdown and restart explicitly	Prevents overlapping or unfinished update loops during repeated restarts.	After 50 game-over/restart cycles, exactly one live `GameCalculator-Thread` is present and game speed remains stable.
P0	Define ownership of mutable game state	Rendering, input, and calculation run on different threads while sharing lists and flags.	A concurrency stress test runs for 10 minutes with no concurrent-modification exceptions, stale-input behavior, or hangs.
P1	Use an application data directory for scores	`scores.txt` currently depends on the process working directory.	Scores save and reload after launching the game from a shortcut, IDE, and a different working directory.
P1	Document setup, controls, and assets	The current README only contains the project name.	A new contributor can build, run, and locate player data using README instructions alone.
P2	Add small automated tests around non-UI rules	Scoring, persistence, and timing rules are currently hard to validate safely.	Tests cover score parsing, ranking limits, power-up expiry, and score/combo calculations.

Recommended Changes

P0 1. Make GameCalculator lifecycle deterministic

Observed path. `SpaceInvadersUI.setGameOver` asks the current calculator to stop. `SpaceInvadersUI.restartGame` immediately creates and starts a new calculator. The inspected

restart path does not wait for the previous worker to finish or clear its reference before replacement.

Proposed implementation. Give the worker a bounded shutdown contract: signal cancellation, interrupt its wait, and join it before a replacement is started. Keep creation and replacement in one lifecycle method so the panel can never own two workers. Make restart idempotent while a restart is already in progress.

Why first. A lingering loop can update the new game state at the same time as the new loop, causing inconsistent movement, collision handling, spawns, or score changes.

- Start the calculator once after the panel is initialized.
- On game over, stop the worker without blocking the Swing event-dispatch thread for an unbounded time.
- On restart, verify the prior worker has exited before starting a replacement.
- Expose a package-private lifecycle seam so this behavior can be tested without rendering a window.

P0 2. Establish a thread-ownership model for game state

Observed path. `GameCalculator` mutates collections and state on its own thread. Swing painting and key listeners run on the event-dispatch thread. Several collection operations are synchronized on the game panel, but rendering reads and scalar state do not consistently follow one synchronization strategy.

Proposed implementation. Choose one model and apply it consistently. The preferred model is to perform all state mutation on the Swing event-dispatch thread using a single Swing timer; it removes cross-thread ownership for a small arcade game. If the worker remains, use a dedicated immutable render snapshot or a single state lock for all read/write access, and use `volatile` or equivalent synchronization for cross-thread flags.

Decision rule. Prefer the single Swing-timer model unless profiling shows it cannot hold the target frame rate. The current 20 ms cadence is modest, and the simpler model reduces race-condition risk substantially.

P1 3. Make score persistence independent of launch location

Observed path. `ScoreFileHandler` opens a relative `scores.txt`. This means the leaderboard location changes with the current working directory, and a packaged game may write to an unexpected or non-writable directory.

Proposed implementation. Resolve a platform-appropriate user data directory, create it on first use, and store scores there through `java.nio.file.Path`. Use a temporary file and atomic move when supported so a crash cannot truncate the leaderboard. Keep malformed-row handling, but surface a concise in-game warning when persistence is unavailable.

P1 4. Turn the README into a usable project guide

Observed path. The README provides only the project name, while the application depends on a specific Java runtime and uses resource files relative to the project.

Proposed contents.

- Java version, compile/run commands, and VS Code launch instructions.
- Controls: Left/Right, Space, P, R, Enter, and the starter-screen difficulty key.
- Resource directory expectations and instructions for adding a theme safely.
- Leaderboard data location and reset procedure.
- A brief architecture map: UI/input, calculation, audio, scoring, and assets.

P2 5. Add a narrow test foundation before more mechanics

Introduce a standard Java test runner and extract pure rule calculations from UI state where necessary. Begin with score parsing and ordering, score-file recovery, combo multiplier rules, power-up expiration, and difficulty calculations. Avoid pixel-level Swing tests until the game-state boundary is stable.

Implementation Sequence

1. Write a regression test or diagnostic that proves one active calculator exists after restart.
2. Implement lifecycle shutdown and confirm repeated restart behavior.
3. Adopt the chosen thread-ownership model and run a focused concurrency stress test.
4. Migrate score storage with a one-time fallback import from the legacy project-local file.
5. Add README documentation and tests for persistence and gameplay rules.

Out of Scope for This Pass

New game modes, multiplayer, additional bosses, artwork, and broad visual redesign should wait. They increase the number of moving parts while the current lifecycle and state-sharing risks remain unverified.

Evidence reviewed: Main startup configuration, SpaceInvadersUI initialization/game-over/restart paths, GameCalculator loop, ListenerActions input handling, ScoreFileHandler persistence, and README.